

CT60A2600-TIIMI-A

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 jono Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 pSeuraava	5
3.1.2.2 pSolmu	5
3.2 puu Struct Reference	6
3.2.1 Detailed Description	6
3.2.2 Member Data Documentation	6
3.2.2.1 aNimi	6
3.2.2.2 iArvo	6
3.2.2.3 iPituus	6
3.2.2.4 pOikea	7
3.2.2.5 pVasen	7
3.3 RBSolmu Struct Reference	7
3.3.1 Detailed Description	7
3.3.2 Member Data Documentation	7
3.3.2.1 aNimi	7
3.3.2.2 iArvo	8
3.3.2.3 iVariBitti	8
3.3.2.4 pOikea	8
3.3.2.5 pVanhempi	8
3.3.2.6 pVasen	8
3.4 tiedot Struct Reference	8
3.4.1 Detailed Description	9
3.4.2 Member Data Documentation	9
3.4.2.1 aSukunimi	9
3.4.2.2 iYhteensa	9
3.4.2.3 pEdellinen	9
3.4.2.4 pSeuraava	9
4 File Documentation	11
4.1 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/binaaripuu.c File Reference	11
4.1.1 Function Documentation	12
4.1.1.1 kirjoitaBinaaripuu()	12
4.1.1.2 kirjoitaTiedostoon()	12
4.1.1.3 lisaaSolmu()	13

4.1.1.4 luoPuu()	14
4.1.1.5 nimenArvo()	15
4.1.1.6 oikeaPuoli()	16
4.1.1.7 onkoLuku()	16
4.1.1.8 paivitaPuu()	17
4.1.1.9 poistaSolmu()	18
4.1.1.10 puunPituus()	19
4.1.1.11 suurempiLukuVertailu()	19
4.1.1.12 tasapainoitaPuu()	20
4.1.1.13 vapautaMuistiPuu()	20
4.1.1.14 varaaMuistiaPuulle()	21
4.1.1.15 vasenPuoli()	21
4.2 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/binaaripuu.h File Reference	22
4.2.1 Typedef Documentation	23
4.2.1.1 PUU	23
4.2.1.2 RBSOLMU	23
4.2.2 Function Documentation	23
4.2.2.1 kierraOikealle()	24
4.2.2.2 kierraVasemmalle()	24
4.2.2.3 kirjoitaBinaaripuu()	24
4.2.2.4 kirjoitaRB()	25
4.2.2.5 kirjoitaRBTiedostoon()	25
4.2.2.6 kirjoitaTiedostoon()	26
4.2.2.7 korjaaLisays()	26
4.2.2.8 lisaaRBSolmu()	27
4.2.2.9 lisaaSolmu()	27
4.2.2.10 luoPuu()	28
4.2.2.11 luoRBPuu()	29
4.2.2.12 nimenArvo()	30
4.2.2.13 oikeaPuoli()	31
4.2.2.14 onkoLuku()	31
4.2.2.15 paivitaPuu()	32
4.2.2.16 poistaSolmu()	33
4.2.2.17 puunPituus()	34
4.2.2.18 suurempiLukuVertailu()	34
4.2.2.19 tasapainoitaPuu()	35
4.2.2.20 vapautaMuistiPuu()	35
4.2.2.21 vapautaMuistiRB()	36
4.2.2.22 varaaMuistiaPuulle()	36
4.2.2.23 varaaMuistiaRB()	37
4.2.2.24 vasenPuoli()	37
4.3 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/haut.c File Reference	38

4.3.1 Function Documentation	38
4.3.1.1 binaarihaku()	38
4.3.1.2 leveyshaku()	39
4.3.1.3 syvyyshaku()	40
4.3.1.4 tarkistaLoytvykoSyvyysaulla()	41
4.3.1.5 vapautaMuistiJono()	41
4.3.1.6 varaaMuistiaJonolle()	43
4.4 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/haut.h File Reference	43
4.4.1 Typedef Documentation	44
4.4.1.1 JONO	44
4.4.2 Function Documentation	44
4.4.2.1 binaarihaku()	44
4.4.2.2 leveyshaku()	45
4.4.2.3 syvyyshaku()	46
4.4.2.4 tarkistaLoytvykoSyvyysaulla()	47
4.4.2.5 vapautaMuistiJono()	47
4.4.2.6 varaaMuistiaJonolle()	49
4.5 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/linkitettylista.c File Reference	50
4.5.1 Function Documentation	50
4.5.1.1 kirjoitaTiedostoAlustaLoppuun()	50
4.5.1.2 kirjoitaTiedostoLopustaAlkuun()	51
4.5.1.3 lue()	52
4.5.1.4 vapautaMuisti()	53
4.5.1.5 varaaMuistia()	53
4.6 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/linkitettylista.h File Reference	54
4.6.1 Typedef Documentation	55
4.6.1.1 TIEDOT	55
4.6.2 Function Documentation	55
4.6.2.1 kirjoitaTiedostoAlustaLoppuun()	55
4.6.2.2 kirjoitaTiedostoLopustaAlkuun()	56
4.6.2.3 lue()	56
4.6.2.4 vapautaMuisti()	57
4.6.2.5 varaaMuistia()	58
4.7 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/paaohjelma.c File Reference	59
4.7.1 Function Documentation	59
4.7.1.1 main()	59
4.8 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/punamustapuu.c File Reference	60
4.8.1 Function Documentation	60
4.8.1.1 kierraOikealle()	60
4.8.1.2 kierraVasemmalle()	61
4.8.1.3 kirjoitaRB()	61
4.8.1.4 kirjoitaRBTiedostoon()	61

4.8.1.5 korjaaLisays()	62
4.8.1.6 lisaaRBSolmu()	63
4.8.1.7 luoRBPuu()	63
4.8.1.8 vapautaMuistiRB()	64
4.8.1.9 varaaMuistiaRB()	64
4.9 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/yleiset.c File Reference	65
4.9.1 Function Documentation	65
4.9.1.1 binaaripuuValikko()	65
4.9.1.2 kysyArvo()	66
4.9.1.3 kysyNimi()	66
4.9.1.4 listaValikko()	67
4.9.1.5 mainBinaaripuu()	67
4.9.1.6 mainLista()	69
4.9.1.7 valikko()	69
4.10 /home/noora/c-projekti/CT60A2600-Tiimi-A/src/yleiset.h File Reference	70
4.10.1 Macro Definition Documentation	70
4.10.1.1 LEN	70
4.10.2 Function Documentation	71
4.10.2.1 binaaripuuValikko()	71
4.10.2.2 kysyArvo()	71
4.10.2.3 kysyNimi()	72
4.10.2.4 listaValikko()	72
4.10.2.5 mainBinaaripuu()	73
4.10.2.6 mainLista()	74
4.10.2.7 valikko()	75

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

jono		5
puu		6
RBSolmu		7
tiedot		8

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ binaaripuu.c	11
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ binaaripuu.h	22
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ haut.c	38
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ haut.h	43
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ linkitettylista.c	50
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ linkitettylista.h	54
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ paaohjelma.c	59
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ punamustapuu.c	60
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ yleiset.c	65
/home/sofia_toropainen/CT60A2600-Tiimi-A/src/ yleiset.h	70

Chapter 3

Class Documentation

3.1 jono Struct Reference

```
#include <haut.h>
```

Public Attributes

- [PUU](#) * [pSolmu](#)
- struct [jono](#) * [pSeuraava](#)

3.1.1 Detailed Description

Definition at line 5 of file haut.h.

3.1.2 Member Data Documentation

3.1.2.1 pSeuraava

```
struct jono* jono::pSeuraava
```

Definition at line 7 of file haut.h.

3.1.2.2 pSolmu

```
PUU* jono::pSolmu
```

Definition at line 6 of file haut.h.

The documentation for this struct was generated from the following file:

- /home/sofia_toropainen/CT60A2600-Tiimi-A/src/[haut.h](#)

3.2 puu Struct Reference

```
#include <binaaripuu.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iPituus](#)
- struct [puu](#) * [pOikea](#)
- struct [puu](#) * [pVasen](#)

3.2.1 Detailed Description

Definition at line 5 of file binaaripuu.h.

3.2.2 Member Data Documentation

3.2.2.1 aNimi

```
char puu::aNimi [LEN]
```

Definition at line 6 of file binaaripuu.h.

3.2.2.2 iArvo

```
int puu::iArvo
```

Definition at line 7 of file binaaripuu.h.

3.2.2.3 iPituus

```
int puu::iPituus
```

Definition at line 8 of file binaaripuu.h.

3.2.2.4 pOikea

```
struct puu* puu::pOikea
```

Definition at line 9 of file binaaripuu.h.

3.2.2.5 pVasen

```
struct puu* puu::pVasen
```

Definition at line 10 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- /home/sofia_toropainen/CT60A2600-Tiimi-A/src/[binaaripuu.h](#)

3.3 RBSolmu Struct Reference

```
#include <binaaripuu.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iVariBitti](#)
- struct [RBSolmu](#) * [pOikea](#)
- struct [RBSolmu](#) * [pVasen](#)
- struct [RBSolmu](#) * [pVanhempi](#)

3.3.1 Detailed Description

Definition at line 13 of file binaaripuu.h.

3.3.2 Member Data Documentation

3.3.2.1 aNimi

```
char RBSolmu::aNimi [LEN]
```

Definition at line 14 of file binaaripuu.h.

3.3.2.2 iArvo

```
int RBSolmu::iArvo
```

Definition at line 15 of file binaaripuu.h.

3.3.2.3 iVariBitti

```
int RBSolmu::iVariBitti
```

Definition at line 16 of file binaaripuu.h.

3.3.2.4 pOikea

```
struct RBSolmu* RBSolmu::pOikea
```

Definition at line 17 of file binaaripuu.h.

3.3.2.5 pVanhempi

```
struct RBSolmu* RBSolmu::pVanhempi
```

Definition at line 19 of file binaaripuu.h.

3.3.2.6 pVasen

```
struct RBSolmu* RBSolmu::pVasen
```

Definition at line 18 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- /home/sofia_toropainen/CT60A2600-Tiimi-A/src/[binaaripuu.h](#)

3.4 tiedot Struct Reference

```
#include <linkitettylista.h>
```

Public Attributes

- char `aSukunimi` [`LEN`]
- int `iYhteensa`
- struct `tiedot` * `pSeuraava`
- struct `tiedot` * `pEdellinen`

3.4.1 Detailed Description

Definition at line 5 of file `linkitettylista.h`.

3.4.2 Member Data Documentation

3.4.2.1 `aSukunimi`

```
char tiedot::aSukunimi[LEN]
```

Definition at line 6 of file `linkitettylista.h`.

3.4.2.2 `iYhteensa`

```
int tiedot::iYhteensa
```

Definition at line 7 of file `linkitettylista.h`.

3.4.2.3 `pEdellinen`

```
struct tiedot* tiedot::pEdellinen
```

Definition at line 9 of file `linkitettylista.h`.

3.4.2.4 `pSeuraava`

```
struct tiedot* tiedot::pSeuraava
```

Definition at line 8 of file `linkitettylista.h`.

The documentation for this struct was generated from the following file:

- `/home/sofia_toropainen/CT60A2600-Tiimi-A/src/linkitettylista.h`

Chapter 4

File Documentation

4.1 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/binaaripuu.c File Reference

```
#include "binaaripuu.h"
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- **PUU * varaaMuistiaPuulle** (char *pSolmu, int iArvo)
Muistin varaaminen puu structin alkioille.
- **PUU * vapautaMuistiPuu** (PUU *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- **PUU * lisaaSolmu** (PUU *pAlku, char *pSolmu, int iArvo)
Lisaa solmun puuhun.
- **PUU * luoPuu** (char *pNimi, PUU *pJuuriSolmu)
Luetaan tiedosto ja luodaan puu rakenne.
- void **kirjoitaBinaaripuu** (char *pNimi, PUU *pAlku)
Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.
- void **kirjoitaTiedostoon** (char *pNimi, PUU *pAlku)
Kirjoittaa binaaripuun tiedostoon.
- int **tasapainoitaPuu** (PUU *pAlku)
Palauttaa solmun tasapainokertoimen.
- **PUU * oikeaPuoli** (PUU *pAlku)
Oikean puolen kierto.
- **PUU * vasenPuoli** (PUU *pAlku)
Vasemman puolen kierto.
- int **puunPituus** (PUU *pAlku)
Hakee solmun korkeuden.
- int **suurempiLukuVertailu** (int iLuku1, int iLuku2)
Vertaa kahta lukua ja palauttaa suuremman.

- `PUU * poistaSolmu` (char *pNimi, `PUU *pJuuriSolmu`)
Lehtisolmun poistaminen.
- int `onkoLuku` (char *pNimi)
Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.
- int `nimenArvo` (char *pNimi, `PUU *pJuuriSolmu`)
Etsii nimen perusteella sen arvon.
- `PUU * paivitaPuu` (`PUU *pJuuriSolmu`)
Tasapainotetaan puu poiston jälkeen.

4.1.1 Function Documentation

4.1.1.1 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku )
```

Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	

Definition at line 176 of file binaaripuu.c.

```
176 {
177     /*if (pAlku == NULL) { //Noora kommentoi pois 19.3.2026 klo. 13.08
178         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
179         return;
180     }*/
181     if (pAlku != NULL) {
182         kirjoitaTiedostoon(pNimi, pAlku);
183         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
184         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
185     }
186     return;
187 }
```

4.1.1.2 kirjoitaTiedostoon()

```
void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku )
```

Kirjoittaa binaaripuun tiedostoon.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 195 of file binaaripuu.c.

```

195                                     {
196     FILE *Tiedosto = NULL;
197
198     if (pAlku == NULL) {
199         return;
200     }
201
202     /* Tiedoston avaaminen. */
203     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
204         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
205         exit(0);
206     }
207     /* Tiedostoon kirjoittaminen. */
208     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
209
210     /* Tiedoston sulkeminen. */
211     fclose(Tiedosto);
212     return;
213 }
```

4.1.1.3 lisaaSolmu()

```

PUU* lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iArvo )
```

Lisaa solmun puuhun.

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iArvo</i>	Solmun arvo.

Returns

PUU*

Definition at line 58 of file binaaripuu.c.

```

58                                     {
59     int iVertailu = 0;
60
61     // Varataan muistia, jos muisti tyhjä.
62     if (pAlku == NULL) {
63         return varaaMuistiaPuulle(pSolmu, iArvo);
64     }
65
66     iVertailu = strcmp(pSolmu, pAlku->aNimi);
67
68     // Solmujen lisääminen.
69     if (iArvo < pAlku->iArvo) {
70         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
71     } else if (iArvo > pAlku->iArvo) {
72         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
73     }
74     // Testataan, ovatko arvot samat.
75     } else if (iArvo == pAlku->iArvo) {
76         if (iVertailu < 0) {
77             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
78         } else if (iVertailu > 0) {
79             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
80         }
81     }
```

```

81     }
82
83     // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
84     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
85     int tasapaino = tasapainoitaPuu(pAlku);
86
87     // vasen vasen tasapainotus
88     if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
89         return (oikeaPuoli(pAlku));
90     }
91
92     // vasen oikea tasapainotus
93     if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
94         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
95         return (oikeaPuoli(pAlku));
96     }
97
98     // oikea vasen tasapainotus
99     if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
100         pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
101         return (vasenPuoli(pAlku));
102     }
103
104     // oikea oikea tasapainotus
105     if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
106         return (vasenPuoli(pAlku));
107     }
108
109     return (pAlku);
110 }

```

4.1.1.4 luoPuu()

```

PUU* luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Luetaan tiedosto ja luodaan puu rakenne.

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 119 of file binaaripuu.c.

```

119
120     FILE *Tiedosto = NULL;
121     char aRivi[LEN] = "";
122     int iOtsikko = 0;
123     char *p1 = NULL, *p2 = NULL;
124     int iArvo = 0;
125
126     // Tiedoston avaus
127     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
128         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
129         exit(0);
130     }
131
132     // Tiedoston lukeminen
133     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
134         aRivi[strlen(aRivi) - 1] = '\0';

```

```

135
136     // Ohitetaan otsikko
137     if (iOtsikko == 0) {
138         iOtsikko++;
139         continue;
140     }
141
142     // Pilkotaan rivit
143     if ((p1 = strtok(aRivi, ";")) == NULL) {
144         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
145         exit(0);
146     }
147
148     if ((p2 = strtok(NULL, ";")) == NULL) {
149         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
150         exit(0);
151     }
152
153     iArvo = atoi(p2);
154
155     // Maaritetaan juuri solmu jos sitä ei ole määritetty.
156     if (pJuuriSolmu == NULL) {
157         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
158         iOtsikko++;
159         continue;
160     }
161
162     pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
163 }
164
165 // Suljetaan tiedosto
166 fclose(Tiedosto);
167 return (pJuuriSolmu);
168 }

```

4.1.1.5 nimenArvo()

```

int nimenArvo (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Etsii nimen perusteella sen arvon.

Parameters

<i>pNimi</i>	Poistettava nimi merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.

Returns

int Palauttaa 0 tai 1, riippuen siitä, löytyykö nimen nimen pohjalta arvoa.

Definition at line 400 of file binaaripuu.c.

```

400     {
401         if (pJuuriSolmu == NULL) {
402             return (-1);
403         }
404
405         if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
406             int iArvo = pJuuriSolmu->iArvo;
407             return (iArvo);
408         }
409
410         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
411         if (iArvo != -1) {
412             return (iArvo);
413         } else {
414             int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);

```

```

415         return (iArvo);
416     }
417 }

```

4.1.1.6 oikeaPuoli()

```

PUU* oikeaPuoli (
    PUU * pAlku )

```

Oikean puolen kierto.

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 237 of file binaaripuu.c.

```

237     {
238         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
239         if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
240             return (pAlku);
241         }
242
243         // Muuttujien ja vakioiden alustaminen
244         PUU *pMuuttuja1 = pAlku->pVasen;
245         PUU *pMuuttuja2 = pMuuttuja1->pOikea;
246
247         // Sijoitetaan arvoja
248         pMuuttuja1->pOikea = pAlku;
249         pAlku->pVasen = pMuuttuja2;
250
251         // Verrataan ja sijoitetaan suuremmat korkeudet
252         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
253         pMuuttuja1->iPituus =
254             suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
255
256         return pMuuttuja1;
257     }

```

4.1.1.7 onkoLuku()

```

int onkoLuku (
    char * pNimi )

```

Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.

Ilmoittaa, onko käyttäjän antama numero vai ei.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
--------------	---

Returns

int Palauttaa 0 tai 1, riippuen siitä onko käyttäjän syöte numero.

Definition at line 379 of file binaaripuu.c.

```

379      {
380      int tosi = 0;
381      char *ptr = pNimi;
382
383      while (*ptr) {
384          if (!isdigit(*ptr)) {
385              return tosi;
386          }
387          ptr++;
388      }
389      tosi = 1;
390      return (tosi);
391  }
```

4.1.1.8 paivitaPuu()

```

PUU* paivitaPuu (
    PUU * pJuuriSolmu )
```

Tasapainotetaan puu poiston jälkeen.

Parameters

<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.
--------------------	--------------------------------

Returns

PUU* palauttaa tasapinotetun solmun.

Definition at line 425 of file binaaripuu.c.

```

425      {
426      // Puun korkeuden päivittäminen.
427      if (pJuuriSolmu == NULL) {
428          return (pJuuriSolmu);
429      }
430
431      int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
432      int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
433      pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
434
435      // Puun tasapanottaminen.
436      int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
437      // Vasen-vasen tapaus.
438      if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
439          return oikeaPuoli(pJuuriSolmu);
440      }
441
442      // Oikea-oikea tapaus.
443      if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
444          return vasenPuoli(pJuuriSolmu);
445      }
446
447      // Vasen-oikea tapaus.
448      if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
449          pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
450          return oikeaPuoli(pJuuriSolmu);
451      }
452
453      // Oikea-vasen tapaus.
454      if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
455          pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
456          return vasenPuoli(pJuuriSolmu);
```

```

457     }
458
459     return (pJuuriSolmu);
460 }

```

4.1.1.9 poistaSolmu()

```

PUU* poistaSolmu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Lehtisolmun poistaminen.

Poistaa lehtisolmun joko numeroarvon tai nimen perusteella.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

pJuuriSolmu Uusi osoitin puun solmuun, NULL jos puu tyhjeni.

Definition at line 334 of file binaaripuu.c.

```

334                                     {
335     int iArvo = 0;
336
337     if (pJuuriSolmu == NULL) {
338         printf("Solmua ei löytynyt.\n");
339         return (pJuuriSolmu);
340     }
341
342     // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
343     if (onkoLuku(pNimi)) {
344         iArvo = atoi(pNimi);
345     } else {
346         iArvo = nimenArvo(pNimi, pJuuriSolmu);
347     }
348
349     // Lehtisolmun poistaminen.
350     if (iArvo < pJuuriSolmu->iArvo) {
351         pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
352     } else if (iArvo > pJuuriSolmu->iArvo) {
353         pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
354     } else {
355         if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
356             free(pJuuriSolmu);
357             pJuuriSolmu = NULL;
358             printf("Solmu poistettu.\n");
359             return (pJuuriSolmu);
360         } else {
361             printf("Voit poistaa vain lehtisolmun.\n");
362             return (pJuuriSolmu);
363         }
364     }
365
366     pJuuriSolmu = paivitaPuu(pJuuriSolmu);
367     return (pJuuriSolmu);
368 }

```


4.1.1.10 puunPituus()

```
int puunPituus (
    PUU * pAlku )
```

Hakee solmun korkeuden.

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 293 of file binaaripuu.c.

```
293     {
294         // Tarkistetaan, onko Solmu Null
295         if (pAlku == NULL) {
296             return (0);
297         }
298
299         // Palauttaa solmun korkeuden
300         return (pAlku->iPituus);
301     }
```

4.1.1.11 suurempiLukuVertailu()

```
int suurempiLukuVertailu (
    int iLuku1,
    int iLuku2 )
```

Vertaa kahta lukua ja palauttaa suuremman.

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 310 of file binaaripuu.c.

```
310     {
311         int iPalautus = 0;
312
313         // Verrataan kumpi luvuista on suurempi.
314         if (iLuku1 > iLuku2) {
315             iPalautus = iLuku1;
316         } else {
317             iPalautus = iLuku2;
318         }
319
320         // Palauttaa suuremman luvun.
321         return (iPalautus);
322     }
```

4.1.1.12 tasapainoitaPuu()

```
int tasapainoitaPuu (
    PUU * pAlku )
```

Palauttaa solmun tasapainokertoimen.

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 221 of file binaaripuu.c.

```
221     {
222     // Tarkistetaan, onko Null
223     if (pAlku == NULL) {
224         return (0);
225     }
226
227     // Palautetaan tasapainokerroin
228     return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
229 }
```

4.1.1.13 vapautaMuistiPuu()

```
PUU* vapautaMuistiPuu (
    PUU * pJuuriSolmu )
```

Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 39 of file binaaripuu.c.

```
39     {
40     // Muistin vapauttaminen
41     if (pJuuriSolmu != NULL) {
42         vapautaMuistiPuu(pJuuriSolmu->pVasen);
43         vapautaMuistiPuu(pJuuriSolmu->pOikea);
44         free(pJuuriSolmu);
45     }
46     pJuuriSolmu = NULL;
47     return (pJuuriSolmu);
48 }
```

4.1.1.14 varaaMuistiaPuulle()

```

PUU* varaaMuistiaPuulle (
    char * pSolmu,
    int iArvo )

```

Muistin varaaminen puu structin alkioille.

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iArvo</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 16 of file binaaripuu.c.

```

16                                     {
17     PUU *pUusi = NULL;
18
19     // Muistin varaaminen
20     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
21         perror("Muistin varaus epäonnistui, lopetetaan");
22         exit(0);
23     }
24
25     strcpy(pUusi->aNimi, pSolmu);
26     pUusi->iArvo = iArvo;
27     pUusi->iPituus = 1;
28     pUusi->pVasen = NULL;
29     pUusi->pOikea = NULL;
30     return (pUusi);
31 }

```

4.1.1.15 vasenPuoli()

```

PUU* vasenPuoli (
    PUU * pAlku )

```

Vasemman puolen kierto.

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 265 of file binaaripuu.c.

```

265                                     {
266     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL

```

```

267     if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
268         return (pAlku);
269     }
270
271     // Muuttujien ja vakioiden alustaminen
272     PUU *pMuuttujal = pAlku->pOikea;
273     PUU *pMuuttuja2 = pMuuttujal->pVasen;
274
275     // Sijoitetetaan arvoja
276     pMuuttujal->pVasen = pAlku;
277     pAlku->pOikea = pMuuttuja2;
278
279     // Verrataan ja sijoitetaan suuremmat korkeudet
280     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
281     pMuuttujal->iPituus =
282         suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
283
284     return pMuuttujal;
285 }

```

4.2 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/binaaripuu.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [puu](#)
- struct [RBSolmu](#)

Typedefs

- typedef struct [puu](#) [PUU](#)
- typedef struct [RBSolmu](#) [RBSOLMU](#)

Functions

- [PUU](#) * [varaaMuistiaPuulle](#) (char *pSolmu, int iArvo)
Muistin varaaminen puu structin alkioille.
- [PUU](#) * [vapautaMuistiPuu](#) ([PUU](#) *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- [PUU](#) * [lisaaSolmu](#) ([PUU](#) *pAlku, char *pSolmu, int iArvo)
Lisaa solmun puuhun.
- [PUU](#) * [luoPuu](#) (char *pNimi, [PUU](#) *pJuuriSolmu)
Luetaan tiedosto ja luodaan puu rakenne.
- int [tasapainoitaPuu](#) ([PUU](#) *pAlku)
Palauttaa solmun tasapainokertoimen.
- [PUU](#) * [oikeaPuoli](#) ([PUU](#) *pAlku)
Oikean puolen kierto.
- [PUU](#) * [vasenPuoli](#) ([PUU](#) *pAlku)
Vasemman puolen kierto.
- int [puunPituus](#) ([PUU](#) *pAlku)
Hakee solmun korkeuden.

- int [suurempiLukuVertailu](#) (int iLuku1, int iLuku2)
Vertaa kahta lukua ja palauttaa suuremman.
- [PUU](#) * [poistaSolmu](#) (char *pNimi, [PUU](#) *pJuuriSolmu)
Lehtisolmun poistaminen.
- int [onkoLuku](#) (char *pNimi)
Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.
- int [nimenArvo](#) (char *pNimi, [PUU](#) *pJuuriSolmu)
Etsii nimen perusteella sen arvon.
- [PUU](#) * [paivitaPuu](#) ([PUU](#) *pJuuriSolmu)
Tasapainotetaan puu poiston jälkeen.
- void [kirjoitaBinaaripuu](#) (char *pNimi, [PUU](#) *pAlku)
Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.
- void [kirjoitaTiedostoon](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pAlku)
Kirjoittaa binaaripuun tiedostoon.
- [RBSOLMU](#) * [varaaMuistiaRB](#) (char *pNimi, int iArvo)
Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.
- [RBSOLMU](#) * [vapautaMuistiRB](#) ([RBSOLMU](#) *pJuuriSolmu)
- [RBSOLMU](#) * [lisaaRBSolmu](#) ([RBSOLMU](#) *pJuuriSolmu, [RBSOLMU](#) *pUusi)
- [RBSOLMU](#) * [luoRBPuu](#) ([RBSOLMU](#) *pJuuriSolmu, char *pNimi)
- void [kierraVasemmalle](#) ([RBSOLMU](#) **pJuuriSolmu, [RBSOLMU](#) *pSolmu)
- void [kierraOikealle](#) ([RBSOLMU](#) **pJuuriSolmu, [RBSOLMU](#) *pSolmu)
- void [korjaaLisays](#) ([RBSOLMU](#) **pJuuriSolmu, [RBSOLMU](#) *pUusi)
- void [kirjoitaRBTiedostoon](#) (char *pNimi, [RBSOLMU](#) *pJuuriSolmu)
- void [kirjoitaRB](#) (char *pNimi, [RBSOLMU](#) *pJuuriSolmu)

4.2.1 Typedef Documentation

4.2.1.1 PUU

```
typedef struct puu PUU
```

4.2.1.2 RBSOLMU

```
typedef struct RBSolmu RBSOLMU
```

4.2.2 Function Documentation

4.2.2.1 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 130 of file punamustapuu.c.

```
130 {
131     RBSOLMU *pVasenLapsi = pSolmu->pVasen;
132     pSolmu->pVasen = pVasenLapsi->pOikea;
133
134     if (pSolmu->pVasen != NULL)
135         pSolmu->pVasen->pVanhempi = pSolmu;
136
137     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
138
139     if (pSolmu->pVanhempi == NULL)
140         *pJuurisolmu = pVasenLapsi;
141     else if (pSolmu == pSolmu->pVanhempi->pVasen)
142         pSolmu->pVanhempi->pVasen = pVasenLapsi;
143     else
144         pSolmu->pVanhempi->pOikea = pVasenLapsi;
145
146     pVasenLapsi->pOikea = pSolmu;
147     pSolmu->pVanhempi = pVasenLapsi;
148 }
```

4.2.2.2 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 109 of file punamustapuu.c.

```
109 {
110     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
111     pSolmu->pOikea = pOikeaLapsi->pVasen;
112
113     if (pSolmu->pOikea != NULL)
114         pSolmu->pOikea->pVanhempi = pSolmu;
115
116     // Solmun oikea lapsi nousee ylös puussa
117     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
118
119     if (pSolmu->pVanhempi == NULL)
120         *pJuurisolmu = pOikeaLapsi;
121     else if (pSolmu == pSolmu->pVanhempi->pVasen)
122         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
123     else
124         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
125
126     pOikeaLapsi->pVasen = pSolmu;
127     pSolmu->pVanhempi = pOikeaLapsi;
128 }
```

4.2.2.3 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku )
```

Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	

Definition at line 176 of file binaaripuu.c.

```

176                                     {
177     /*if (pAlku == NULL) { //Noora kommentoi pois 19.3.2026 klo. 13.08
178         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
179         return;
180     }*/
181     if (pAlku != NULL) {
182         kirjoitaTiedostoon(pNimi, pAlku);
183         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
184         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
185     }
186     return;
187 }
```

4.2.2.4 kirjoitaRB()

```

void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 222 of file punamustapuu.c.

```

222                                     {
223     /*if (pJuurisolmu == NULL) {
224         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
225         return;
226     }*/
227     if (pJuurisolmu != NULL) {
228         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
229         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
230         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
231     }
232     return;
233 }
```

4.2.2.5 kirjoitaRBTiedostoon()

```

void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 201 of file punamustapuu.c.

```

201                                     {
202     FILE *Tiedosto = NULL;
203
204     if (pJuurisolmu == NULL) {
205         return;
206     }
207
208     /* Tiedoston avaaminen. */
209     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
210         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
211         exit(0);
212     }
213     /* Tiedostoon kirjoittaminen. */
214     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
215
216     /* Tiedoston sulkeminen. */
217     fclose(Tiedosto);
218     printf("Tiedosto '%s' kirjoitettu.\n", pNimi);
219     return;
220 }
```

4.2.2.6 kirjoitaTiedostoon()

```
void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku )
```

Kirjoittaa binaaripuun tiedostoon.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 195 of file binaaripuu.c.

```
195                                     {
196     FILE *Tiedosto = NULL;
197
198     if (pAlku == NULL) {
199         return;
200     }
201
202     /* Tiedoston avaaminen. */
203     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
204         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
205         exit(0);
206     }
207     /* Tiedostoon kirjoittaminen. */
208     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
209
210     /* Tiedoston sulkeminen. */
211     fclose(Tiedosto);
212     return;
213 }
```

4.2.2.7 korjaaLisays()

```
void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi )
```

Definition at line 150 of file punamustapuu.c.

```
150                                     {
151     // Ilman p-alkua, koska pVanhempi->pVanhempi oli sekava
152     RBSOLMU *vanhempi = NULL;
153     RBSOLMU *seta = NULL;
154     RBSOLMU *isoVanhempi = NULL;
155
156     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
157         vanhempi = pUusi->pVanhempi;
158         isoVanhempi = vanhempi->pVanhempi;
159
160         if (vanhempi == isoVanhempi->pVasen) {
161             seta = isoVanhempi->pOikea;
162             if (seta != NULL && seta->iVariBitti == 0) {
163                 vanhempi->iVariBitti = 1;
164                 seta->iVariBitti = 1;
165                 isoVanhempi->iVariBitti = 0;
166                 pUusi = isoVanhempi;
167             } else {
168                 if (pUusi == vanhempi->pOikea) {
169                     pUusi = vanhempi;
170                     kierraVasemmalle(pJuurisolmu, pUusi);
171                     vanhempi = pUusi->pVanhempi;
172                 }
173                 vanhempi->iVariBitti = 1;
174                 isoVanhempi->iVariBitti = 0;
175                 kierraOikealle(pJuurisolmu, isoVanhempi);
176             }
177         }
178     }
```



```

177
178     } else {
179         seta = isoVanhempi->pVasen;
180         if (seta != NULL && seta->iVariBitti == 0) {
181             vanhempi->iVariBitti = 1;
182             seta->iVariBitti = 1;
183             isoVanhempi->iVariBitti = 0;
184             pUusi = isoVanhempi;
185         } else {
186             if (pUusi == vanhempi->pVasen) {
187                 pUusi = vanhempi;
188                 kierraOikealle(pJuurisolmu, pUusi);
189                 vanhempi = pUusi->pVanhempi;
190             }
191             vanhempi->iVariBitti = 1;
192             isoVanhempi->iVariBitti = 0;
193             kierraVasemmalle(pJuurisolmu, isoVanhempi);
194         }
195     }
196 }
197 (*pJuurisolmu)->iVariBitti = 1;
198 }

```

4.2.2.8 lisaaRBSolmu()

```

RBSOLMU* lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi )

```

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 38 of file punamustapuu.c.

```

38                                     {
39     int iVertailu = 0;
40
41     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
42     if (pJuurisolmu == NULL) {
43         return pUusi;
44     }
45
46     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
47
48     // Solmujen lisääminen rekursiivisesti
49     if (pUusi->iArvo < pJuurisolmu->iArvo) {
50         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
51         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
52     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
53         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
54         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
55     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
56         if (iVertailu < 0) {
57             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
58             pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
59         } else if (iVertailu > 0) {
60             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
61             pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
62         }
63     }
64     return (pJuurisolmu);
65 }
66
67 }

```

4.2.2.9 lisaaSolmu()

```

PUU* lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iArvo )

```

Lisaa solmun puuhun.

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iArvo</i>	Solmun arvo.

Returns

PUU*

Definition at line 58 of file binaaripuu.c.

```

58                                     {
59     int iVertailu = 0;
60
61     // Varataan muistia, jos muisti tyhjä.
62     if (pAlku == NULL) {
63         return varaaMuistiaPuulle(pSolmu, iArvo);
64     }
65
66     iVertailu = strcmp(pSolmu, pAlku->aNimi);
67
68     // Solmujen lisääminen.
69     if (iArvo < pAlku->iArvo) {
70         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
71     } else if (iArvo > pAlku->iArvo) {
72         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
73     }
74     // Testataan, ovatko arvot samat.
75     } else if (iArvo == pAlku->iArvo) {
76         if (iVertailu < 0) {
77             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
78         } else if (iVertailu > 0) {
79             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
80         }
81     }
82
83     // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
84     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
85     int tasapaino = tasapainoitaPuu(pAlku);
86
87     // vasen vasen tasapainotus
88     if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
89         return (oikeaPuoli(pAlku));
90     }
91
92     // vasen oikea tasapainotus
93     if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
94         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
95         return (oikeaPuoli(pAlku));
96     }
97
98     // oikea vasen tasapainotus
99     if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
100         pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
101         return (vasenPuoli(pAlku));
102     }
103
104     // oikea oikea tasapainotus
105     if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
106         return (vasenPuoli(pAlku));
107     }
108
109     return (pAlku);
110 }

```

4.2.2.10 luoPuu()

```

PUU* luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Luetaan tiedosto ja luodaan puu rakenne.

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 119 of file binaaripuu.c.

```

119                                     {
120     FILE *Tiedosto = NULL;
121     char aRivi[LEN] = "";
122     int iOtsikko = 0;
123     char *p1 = NULL, *p2 = NULL;
124     int iArvo = 0;
125
126     // Tiedoston avaus
127     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
128         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
129         exit(0);
130     }
131
132     // Tiedoston lukeminen
133     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
134         aRivi[strlen(aRivi) - 1] = '\0';
135
136         // Ohitetaan otsikko
137         if (iOtsikko == 0) {
138             iOtsikko++;
139             continue;
140         }
141
142         // Pilkkotaan rivit
143         if ((p1 = strtok(aRivi, ";")) == NULL) {
144             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
145             exit(0);
146         }
147
148         if ((p2 = strtok(NULL, ";")) == NULL) {
149             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
150             exit(0);
151         }
152
153         iArvo = atoi(p2);
154
155         // Maaritetaan juuri solmu jos sitä ei ole määritetty.
156         if (pJuuriSolmu == NULL) {
157             pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
158             iOtsikko++;
159             continue;
160         }
161
162         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
163     }
164
165     // Suljetaan tiedosto
166     fclose(Tiedosto);
167     return (pJuuriSolmu);
168 }

```

4.2.2.11 luoRBPuu()

```

RBSOLMU* luoRBPuu (
    RBSOLMU * pJuuriSolmu,
    char * pNimi )

```

Definition at line 69 of file punamustapuu.c.

```

69                                     {
70     FILE *Tiedosto = NULL;

```

```

71     char aRivi[LEN] = "";
72     int iOtsikko = 0;
73     char *p1 = NULL, *p2 = NULL;
74     int iArvo = 0;
75
76     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
77         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
78         exit(0);
79     }
80
81     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
82         aRivi[strlen(aRivi) - 1] = '\0';
83         if (iOtsikko == 0) {
84             iOtsikko++;
85             continue;
86         }
87
88         if ((p1 = strtok(aRivi, ";")) == NULL) {
89             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
90             exit(0);
91         }
92
93         if ((p2 = strtok(NULL, ";")) == NULL) {
94             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
95             exit(0);
96         }
97
98         iArvo = atoi(p2);
99
100         RBSOLMU *pUusi = varaaMuistiaRB(p1, iArvo);
101         pJuuriSolmu = lisaaRBSolmu(pJuuriSolmu, pUusi);
102         korjaaLisays(&pJuuriSolmu, pUusi);
103     }
104
105     fclose(Tiedosto);
106     return (pJuuriSolmu);
107 }

```

4.2.2.12 nimenArvo()

```

int nimenArvo (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Etsii nimen perusteella sen arvon.

Parameters

<i>pNimi</i>	Poistettava nimi merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.

Returns

int Palauttaa 0 tai 1, riippuen siitä, löytyykö nimen nimen pohjalta arvoa.

Definition at line 400 of file binaaripuu.c.

```

400     {
401         if (pJuuriSolmu == NULL) {
402             return (-1);
403         }
404
405         if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
406             int iArvo = pJuuriSolmu->iArvo;
407             return (iArvo);
408         }
409
410         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
411         if (iArvo != -1) {

```

```

412         return (iArvo);
413     } else {
414         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);
415         return (iArvo);
416     }
417 }

```

4.2.2.13 oikeaPuoli()

```

PUU* oikeaPuoli (
    PUU * pAlku )

```

Oikean puolen kierto.

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 237 of file binaaripuu.c.

```

237     {
238         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
239         if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
240             return (pAlku);
241         }
242
243         // Muuttujien ja vakioiden alustaminen
244         PUU *pMuuttuja1 = pAlku->pVasen;
245         PUU *pMuuttuja2 = pMuuttuja1->pOikea;
246
247         // Sijoitetetaan arvoja
248         pMuuttuja1->pOikea = pAlku;
249         pAlku->pVasen = pMuuttuja2;
250
251         // Verrataan ja sijoitetaan suuremmat korkeudet
252         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
253         pMuuttuja1->iPituus =
254             suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
255
256         return pMuuttuja1;
257 }

```

4.2.2.14 onkoLuku()

```

int onkoLuku (
    char * pNimi )

```

Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.

Ilmoittaa, onko käyttäjän antama numero vai ei.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
--------------	---

Returns

int Palauttaa 0 tai 1, riippuen siitä onko käyttäjän syöte numero.

Definition at line 379 of file binaaripuu.c.

```

379     {
380         int tosi = 0;
381         char *ptr = pNimi;
382
383         while (*ptr) {
384             if (!isdigit(*ptr)) {
385                 return tosi;
386             }
387             ptr++;
388         }
389         tosi = 1;
390         return (tos);
391     }

```

4.2.2.15 paivitaPuu()

```

PUU* paivitaPuu (
    PUU * pJuuriSolmu )

```

Tasapainotetaan puu poiston jälkeen.

Parameters

<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.
--------------------	--------------------------------

Returns

PUU* palauttaa tasapainotetun solmun.

Definition at line 425 of file binaaripuu.c.

```

425     {
426         // Puun korkeuden päivittäminen.
427         if (pJuuriSolmu == NULL) {
428             return (pJuuriSolmu);
429         }
430
431         int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
432         int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
433         pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
434
435         // Puun tasapanottaminen.
436         int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
437         // Vasen-vasen tapaus.
438         if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
439             return oikeaPuoli(pJuuriSolmu);
440         }
441
442         // Oikea-oikea tapaus.
443         if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
444             return vasenPuoli(pJuuriSolmu);
445         }
446
447         // Vasen-oikea tapaus.

```

```

448     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
449         pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
450         return oikeaPuoli(pJuuriSolmu);
451     }
452
453     // Oikea-vasen tapaus.
454     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
455         pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
456         return vasenPuoli(pJuuriSolmu);
457     }
458
459     return (pJuuriSolmu);
460 }

```

4.2.2.16 poistaSolmu()

```

PUU* poistaSolmu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Lehtisolmun poistaminen.

Poistaa lehtisolmun joko numeroarvon tai nimen perusteella.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

pJuuriSolmu Uusi osoitin puun solmuun, NULL jos puu tyhjeni.

Definition at line 334 of file binaaripuu.c.

```

334     {
335         int iArvo = 0;
336
337         if (pJuuriSolmu == NULL) {
338             printf("Solmua ei löytynyt.\n");
339             return (pJuuriSolmu);
340         }
341
342         // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
343         if (onkoLuku(pNimi)) {
344             iArvo = atoi(pNimi);
345         } else {
346             iArvo = nimenArvo(pNimi, pJuuriSolmu);
347         }
348
349         // Lehtisolmun poistaminen.
350         if (iArvo < pJuuriSolmu->iArvo) {
351             pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
352         } else if (iArvo > pJuuriSolmu->iArvo) {
353             pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
354         } else {
355             if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
356                 free(pJuuriSolmu);
357                 pJuuriSolmu = NULL;
358                 printf("Solmu poistettu.\n");
359                 return (pJuuriSolmu);
360             } else {
361                 printf("Voit poistaa vain lehtisolmun.\n");
362                 return (pJuuriSolmu);
363             }
364         }

```

```

365
366     pJuuriSolmu = paivitaPuu(pJuuriSolmu);
367     return (pJuuriSolmu);
368 }

```

4.2.2.17 puunPituus()

```

int puunPituus (
    PUU * pAlku )

```

Hakee solmun korkeuden.

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 293 of file binaaripuu.c.

```

293     {
294         // Tarkistetaan, onko Solmu Null
295         if (pAlku == NULL) {
296             return (0);
297         }
298
299         // Palauttaa solmun korkeuden
300         return (pAlku->iPituus);
301     }

```

4.2.2.18 suurempiLukuVertailu()

```

int suurempiLukuVertailu (
    int iLuku1,
    int iLuku2 )

```

Vertaa kahta lukua ja palauttaa suuremman.

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 310 of file binaaripuu.c.

```

310     {
311         int iPalautus = 0;

```



```

312
313     // Verrataan kumpi luvuista on suurempi.
314     if (iLuku1 > iLuku2) {
315         iPalautus = iLuku1;
316     } else {
317         iPalautus = iLuku2;
318     }
319
320     // Palauttaa suuremman luvun.
321     return (iPalautus);
322 }

```

4.2.2.19 tasapainoitaPuu()

```

int tasapainoitaPuu (
    PUU * pAlku )

```

Palauttaa solmun tasapainokertoimen.

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 221 of file binaaripuu.c.

```

221     {
222     // Tarkistetaan, onko Null
223     if (pAlku == NULL) {
224         return (0);
225     }
226
227     // Palautetaan tasapainokerroin
228     return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
229 }

```

4.2.2.20 vapautaMuistiPuu()

```

PUU* vapautaMuistiPuu (
    PUU * pJuuriSolmu )

```

Muistin vapauttaminen puu structin alkiosta Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 39 of file binaaripuu.c.

```

39                                     {
40     // Muistin vapauttaminen
41     if (pJuuriSolmu != NULL) {
42         vapautaMuistiPuu(pJuuriSolmu->pVasen);
43         vapautaMuistiPuu(pJuuriSolmu->pOikea);
44         free(pJuuriSolmu);
45     }
46     pJuuriSolmu = NULL;
47     return (pJuuriSolmu);
48 }

```

4.2.2.21 vapautaMuistiRB()

```

RBSOLMU* vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu )

```

Definition at line 28 of file punamustapuu.c.

```

28                                     {
29     if (pJuuriSolmu != NULL) {
30         vapautaMuistiRB(pJuuriSolmu->pVasen);
31         vapautaMuistiRB(pJuuriSolmu->pOikea);
32         free(pJuuriSolmu);
33     }
34     pJuuriSolmu = NULL;
35     return (pJuuriSolmu);
36 }

```

4.2.2.22 varaaMuistiaPuulle()

```

PUU* varaaMuistiaPuulle (
    char * pSolmu,
    int iArvo )

```

Muistin varaaminen puu structin alkioille.

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iArvo</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 16 of file binaaripuu.c.

```

16                                     {
17     PUU *pUusi = NULL;
18
19     // Muistin varaaminen
20     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
21         perror("Muistin varaus epäonnistui, lopetetaan");
22         exit(0);
23     }
24
25     strcpy(pUusi->aNimi, pSolmu);
26     pUusi->iArvo = iArvo;
27     pUusi->iPituus = 1;

```

```

28     pUusi->pVasen = NULL;
29     pUusi->pOikea = NULL;
30     return (pUusi);
31 }

```

4.2.2.23 varaaMuistiaRB()

```

RBSOLMU* varaaMuistiaRB (
    char * pNimi,
    int iArvo )

```

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Definition at line 11 of file punamustapuu.c.

```

11                                     {
12     RBSOLMU *pUusi = NULL;
13
14     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
15         perror("Muistin varaus epäonnistui, lopetetaan");
16         exit(0);
17     }
18
19     pUusi->iVariBitti = 0; // uudet aina punaisia
20     strcpy(pUusi->aNimi, pNimi);
21     pUusi->iArvo = iArvo;
22     pUusi->pVasen = NULL;
23     pUusi->pOikea = NULL;
24     pUusi->pVanhempi = NULL;
25     return (pUusi);
26 }

```

4.2.2.24 vasenPuoli()

```

PUU* vasenPuoli (
    PUU * pAlku )

```

Vasemman puolen kierto.

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 265 of file binaaripuu.c.

```

265                                     {
266     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
267     if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
268         return (pAlku);
269     }

```

```

270
271 // Muuttujien ja vakioiden alustaminen
272 PUU *pMuuttujal = pAlku->pOikea;
273 PUU *pMuuttuja2 = pMuuttujal->pVasen;
274
275 // Sijoitetetaan arvoja
276 pMuuttujal->pVasen = pAlku;
277 pAlku->pOikea = pMuuttuja2;
278
279 // Verrataan ja sijoitetaan suuremmat korkeudet
280 pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
281 pMuuttujal->iPituus =
282     suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
283
284 return pMuuttujal;
285 }

```

4.3 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/haut.c File Reference

```

#include "haut.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Functions

- int **syvyyshaku** (char *pNimi, PUU *pAlku, int iArvo)
Tekee syvyysshaun numeroarvon pohjalta.
- void **tarkistaLoytyykoSyvyysshaulla** (char *aNimiKirjoitettava, PUU *pJuuriSolmu, int iArvo)
Tarkistaa, loytyyko etsitty arvo syvyysshaussa.
- void **leveyshaku** (char *pNimi, PUU *pJuuriSolmu, char *pHaku)
Tekee leveyshaun nimen pohjalta.
- JONO * **varaaMuistiaJonolle** (PUU *pSolmu)
Varaa muistia jonolle.
- JONO * **vapautaMuistiJono** (JONO *pAlku)
Vapauttaa jonon varaaman muistin.
- int **binaarihaku** (char *pNimi, PUU *pJuuriSolmu, int iArvo)
Binaarihaku perustuen numeroarvoon.

4.3.1 Function Documentation

4.3.1.1 binaarihaku()

```

int binaarihaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    int iArvo )

```

Binaarihaku perustuen numeroarvoon.

Binaarihaku, joka tehdään käyttäjän antaman numeroarvon perusteella.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

int Palauttaa joko 0 tai 1, riippuen siitä, löytyykö numeroarvo binaaripuusta.

Definition at line 175 of file haut.c.

```

175                                     {
176
177     if (pJuuriSolmu == NULL) {
178         return (0);
179     }
180
181     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
182
183     if (iArvo == pJuuriSolmu->iArvo) {
184         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
185         return (1);
186     } else if (iArvo < pJuuriSolmu->iArvo) {
187         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
188     } else {
189         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
190     }
191 }
```

4.3.1.2 leveyshaku()

```

void leveyshaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    char * pHaku )
```

Tekee leveyshaun nimen pohjalta.

Tekee leveyshaun käyttäjän antaman nimen pohjalta. Kirjoittaa leveyshaun polun käyttäjän maarittelemaan tiedoston.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>pHaku</i>	Osoitin haettavan nimen merkkijonoon.

Returns

void

Definition at line 73 of file haut.c.

```

73                                     {
74     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
75     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
76     JONO *pLoppu = pAlku;
77     JONO *pEdellinen = NULL;
```

```

78     int iVertailu = 0;
79     int iLoytyi = 0;
80
81     if (pJuuriSolmu == NULL) {
82         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
83         return;
84     }
85
86     while (pAlku != NULL) {
87         PUU *ptr = pAlku->pSolmu;
88         kirjoitaTiedostoon(pNimi, ptr);
89
90         iVertailu = strcmp(ptr->aNimi, pHaku);
91
92         if (iVertailu == 0) {
93             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
94             iLoytyi = 1;
95             break;
96         }
97
98         if (ptr->pVasen != NULL) {
99             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
100             pLoppu = pLoppu->pSeuraava;
101         }
102
103         if (ptr->pOikea != NULL) {
104             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
105             pLoppu = pLoppu->pSeuraava;
106         }
107
108         pEdellinen = pAlku;
109         pAlku = pAlku->pSeuraava;
110         free(pEdellinen);
111     }
112
113     if (iLoytyi == 0) {
114         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
115     }
116     pAlku = vapautaMuistiJono(pAlku);
117     return;
118 }

```

4.3.1.3 syvyyshaku()

```

int syvyyshaku (
    char * pNimi,
    PUU * pAlku,
    int iArvo )

```

Tekee syvyysshaun numeroarvon pohjalta.

Tekee syvyysshaun käyttäjän antaman numeroarvon pohjalta. Kirjoittaa syvyysshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Arvo, jota etsitään syvyysshaussa.

Returns

int Palauttaa arvon 0 tai 1, riippuen siitä, löytyykö etsitty arvo.

Definition at line 17 of file haut.c.

```

17     {

```

```

18 // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
19 int iLoytyi = 0;
20
21 if (pAlku == NULL) {
22     return 0;
23 }
24
25 if (pAlku != NULL) {
26     kirjoitaTiedostoon(pNimi, pAlku);
27
28     if (pAlku->iArvo == iArvo) {
29         iLoytyi = 1;
30         printf("Arvo %d löytyi.\n", iArvo);
31     } else {
32         if (iLoytyi == 0) {
33             iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
34         }
35         if (iLoytyi == 0) {
36             iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
37         }
38     }
39 }
40 return (iLoytyi);
41 }

```

4.3.1.4 tarkistaLoytyykoSyvyyshauulla()

```

void tarkistaLoytyykoSyvyyshauulla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo )

```

Tarkistaa, löytyykö etsitty arvo syvyysaussa.

Tarkistaa, löytyykö syvyysaussa etsitty arvo. Jos arvoa ei löydy, tulostaa tiedon siitä.

Parameters

<i>aNimiKirjoitettava</i>	Kayttajan antama kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Syvyysaussa haettava numeroarvo.

Returns

void

Definition at line 54 of file haut.c.

```

54
55     int iLoytyi = 0;
56     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
57     if (iLoytyi == 0) {
58         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
59     }
60 }

```

4.3.1.5 vapautaMuistiJono()

```

JONO* vapautaMuistiJono (
    JONO * pAlku )

```

Vapauttaa jonon varaaman muistin.

Vapauttaa jonon alkioita varten varatun muistin.

Parameters

<i>pAlku</i>	Osoitin jonon ensimmäiseen alkioon.
--------------	-------------------------------------

Returns

pAlku Palauttaa NULL, koska jono on tyhjä.

Definition at line 151 of file haut.c.

```
151                                     {
152     // Jonon muistin vapauttaminen.
153     JONO *ptr = pAlku;
154     JONO *pSeuraava = NULL;
155
156     while (ptr != NULL) {
157         pSeuraava = ptr->pSeuraava;
158         free(ptr);
159         ptr = pSeuraava;
160     }
161     pAlku = NULL;
162     return (pAlku);
163 }
```

4.3.1.6 varaaMuistiaJonolle()

```
JONO* varaaMuistiaJonolle (
    PUU * pSolmu )
```

Varaa muistia jonolle.

Varaa muistia jonon uudelle alkionle leveyshakua varten.

Parameters

<i>pSolmu</i>	Osoitin kasiteltavaan puun solmuun.
---------------	-------------------------------------

Returns

pUusi Osoitin uuteen jonon alkioon.

Definition at line 128 of file haut.c.

```
128                                     {
129     JONO *pUusi = NULL;
130
131     // Muistin varaaminen jonolle.
132     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
133         perror("Muistin varaus epäonnistui, lopetetaan");
134         exit(0);
135     }
136
137     pUusi->pSolmu = pSolmu;
138     pUusi->pSeuraava = NULL;
139
140     return (pUusi);
141 }
```

4.4 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/haut.h File Reference

```
#include "binaaripuu.h"
```

Classes

- struct [jono](#)

Typedefs

- typedef struct [jono](#) [JONO](#)

Functions

- int [syvyyshaku](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Tekee syvyysshaun numeroarvon pohjalta.
- void [tarkistaLoytvykoSyvyysshaulla](#) (char *aNimiKirjoitettava, [PUU](#) *pJuuriSolmu, int iArvo)
Tarkistaa, loytyyko etsitty arvo syvyysshaussa.
- void [leveyshaku](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pAlku, char *pHaku)
Tekee leveyshaun nimen pohjalta.
- [JONO](#) * [varaaMuistiaJonolle](#) ([PUU](#) *pSolmu)
Varaa muistia jonolle.
- [JONO](#) * [vapautaMuistiJono](#) ([JONO](#) *pAlku)
Vapauttaa jonon varaaman muistin.
- int [binaarihaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Binaarihaku perustuen numeroarvoon.

4.4.1 Typedef Documentation

4.4.1.1 JONO

```
typedef struct jono JONO
```

4.4.2 Function Documentation

4.4.2.1 binaarihaku()

```
int binaarihaku (  
    char * pNimi,  
    PUU * pJuuriSolmu,  
    int iArvo )
```

Binaarihaku perustuen numeroarvoon.

Binaarihaku, joka tehdään käyttäjän antaman numeroarvon perusteella.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

int Palauttaa joko 0 tai 1, riippuen siitä, löytyykö numeroarvo binaaripuusta.

Definition at line 175 of file haut.c.

```

175                                     {
176
177     if (pJuuriSolmu == NULL) {
178         return (0);
179     }
180
181     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
182
183     if (iArvo == pJuuriSolmu->iArvo) {
184         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
185         return (1);
186     } else if (iArvo < pJuuriSolmu->iArvo) {
187         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
188     } else {
189         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
190     }
191 }
```

4.4.2.2 leveyshaku()

```

void leveyshaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    char * pHaku )
```

Tekee leveyshaun nimen pohjalta.

Tekee leveyshaun käyttäjän antaman nimen pohjalta. Kirjoittaa leveyshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>pHaku</i>	Osoitin haettavan nimen merkkijonoon.

Returns

void

Definition at line 73 of file haut.c.

```

73                                     {
74     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
75     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
76     JONO *pLoppu = pAlku;
77     JONO *pEdellinen = NULL;
```

```

78     int iVertailu = 0;
79     int iLoytyi = 0;
80
81     if (pJuuriSolmu == NULL) {
82         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
83         return;
84     }
85
86     while (pAlku != NULL) {
87         PUU *ptr = pAlku->pSolmu;
88         kirjoitaTiedostoon(pNimi, ptr);
89
90         iVertailu = strcmp(ptr->aNimi, pHaku);
91
92         if (iVertailu == 0) {
93             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
94             iLoytyi = 1;
95             break;
96         }
97
98         if (ptr->pVasen != NULL) {
99             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
100             pLoppu = pLoppu->pSeuraava;
101         }
102
103         if (ptr->pOikea != NULL) {
104             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
105             pLoppu = pLoppu->pSeuraava;
106         }
107
108         pEdellinen = pAlku;
109         pAlku = pAlku->pSeuraava;
110         free(pEdellinen);
111     }
112
113     if (iLoytyi == 0) {
114         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
115     }
116     pAlku = vapautaMuistiJono(pAlku);
117     return;
118 }

```

4.4.2.3 syvyyshaku()

```

int syvyyshaku (
    char * pNimi,
    PUU * pAlku,
    int iArvo )

```

Tekee syvyysshaun numeroarvon pohjalta.

Tekee syvyysshaun käyttäjän antaman numeroarvon pohjalta. Kirjoittaa syvyysshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Arvo, jota etsitään syvyysshaussa.

Returns

int Palauttaa arvon 0 tai 1, riippuen siitä, löytyykö etsitty arvo.

Definition at line 17 of file haut.c.

```

17     {

```

```

18 // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
19 int iLoytyi = 0;
20
21 if (pAlku == NULL) {
22     return 0;
23 }
24
25 if (pAlku != NULL) {
26     kirjoitaTiedostoon(pNimi, pAlku);
27
28     if (pAlku->iArvo == iArvo) {
29         iLoytyi = 1;
30         printf("Arvo %d löytyi.\n", iArvo);
31     } else {
32         if (iLoytyi == 0) {
33             iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
34         }
35         if (iLoytyi == 0) {
36             iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
37         }
38     }
39 }
40 return (iLoytyi);
41 }

```

4.4.2.4 tarkistaLoytyykoSyvyyshauulla()

```

void tarkistaLoytyykoSyvyyshauulla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo )

```

Tarkistaa, löytyykö etsitty arvo syvyysaussa.

Tarkistaa, löytyykö syvyysaussa etsitty arvo. Jos arvoa ei löydy, tulostaa tiedon siitä.

Parameters

<i>aNimiKirjoitettava</i>	Kayttajan antama kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Syvyysaussa haettava numeroarvo.

Returns

void

Definition at line 54 of file haut.c.

```

54
55     int iLoytyi = 0;
56     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
57     if (iLoytyi == 0) {
58         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
59     }
60 }

```

4.4.2.5 vapautaMuistiJono()

```

JONO* vapautaMuistiJono (
    JONO * pAlku )

```

Vapauttaa jonon varaaman muistin.

Vapauttaa jonon alkioita varten varatun muistin.

Parameters

<i>pAlku</i>	Osoitin jonon ensimmäiseen alkioon.
--------------	-------------------------------------

Returns

pAlku Palauttaa NULL, koska jono on tyhjä.

Definition at line 151 of file haut.c.

```
151                                     {
152     // Jonon muistin vapauttaminen.
153     JONO *ptr = pAlku;
154     JONO *pSeuraava = NULL;
155
156     while (ptr != NULL) {
157         pSeuraava = ptr->pSeuraava;
158         free(ptr);
159         ptr = pSeuraava;
160     }
161     pAlku = NULL;
162     return (pAlku);
163 }
```

4.4.2.6 varaaMuistiaJonolle()

```
JONO* varaaMuistiaJonolle (
    PUU * pSolmu )
```

Varaa muistia jonolle.

Varaa muistia jonon uudelle alkiole leveyshakua varten.

Parameters

<i>pSolmu</i>	Osoitin kasiteltavaan puun solmuun.
---------------	-------------------------------------

Returns

pUusi Osoitin uuteen jonon alkioon.

Definition at line 128 of file haut.c.

```
128                                     {
129     JONO *pUusi = NULL;
130
131     // Muistin varaaminen jonolle.
132     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
133         perror("Muistin varaus epäonnistui, lopetetaan");
134         exit(0);
135     }
136
137     pUusi->pSolmu = pSolmu;
138     pUusi->pSeuraava = NULL;
139
140     return (pUusi);
141 }
```

4.5 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/linkitettylista.c File Reference

```
#include "linkitettylista.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- `TIEDOT * lue` (char *pNimi, `TIEDOT *pAlku`)
Lukee tiedoston.
- `TIEDOT * varaaMuistia` (`TIEDOT *pAlku`, char *pNimi, int iMaara)
Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.
- `TIEDOT * vapautaMuisti` (`TIEDOT *pAlku`)
Vapauttaa linkitetyn listan varaaman muistin.
- void `kirjoitaTiedostoAlustaLoppuun` (char *pNimi, `TIEDOT *pAlku`)
Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.
- void `kirjoitaTiedostoLopustaAlkuun` (char *pNimi, `TIEDOT *pAlku`)
Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.

4.5.1 Function Documentation

4.5.1.1 kirjoitaTiedostoAlustaLoppuun()

```
void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku )
```

Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 129 of file linkitettylista.c.

```
129
130     FILE *Tiedosto = NULL;
131     TIEDOT *ptr = NULL;
132
133     if (pAlku == NULL) {
```



```

134     printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
135     return;
136 }
137
138 // Kirjoitus tiedoston avaaminen
139 if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
140     perror("Tiedoston avaaminen epäonnistui, lopetetaan");
141     exit(0);
142 }
143
144 // Tiedostoon kirjoittaminen
145 ptr = pAlku;
146 while (ptr != NULL) {
147     fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
148     ptr = ptr->pSeuraava;
149 }
150
151 // Tiedoston sulkeminen
152 fclose(Tiedosto);
153 printf("Tiedosto %s kirjoitettu.\n", pNimi);
154 return;
155 }

```

4.5.1.2 kirjoitaTiedostoLopustaAlkuun()

```

void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku )

```

Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 164 of file linkitettylista.c.

```

164
165     FILE *Tiedosto = NULL;
166     TIEDOT *ptr = pAlku;
167
168     if (pAlku == NULL) {
169         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
170         return;
171     }
172
173     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
174         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
175         exit(0);
176     }
177
178     // Mennään listan loppuun.
179     while (ptr->pSeuraava != NULL) {
180         ptr = ptr->pSeuraava;
181     }
182
183     // Lopusta alkuun, kirjoitetaan tiedostoon.
184     while (ptr != NULL) {
185         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
186         ptr = ptr->pEdellinen;
187     }
188
189     fclose(Tiedosto);
190     printf("Tiedosto %s kirjoitettu.\n", pNimi);

```

```

191     return;
192 }

```

4.5.1.3 lue()

```

TIEDOT* lue (
    char * pNimi,
    TIEDOT * pAlku )

```

Lukee tiedoston.

Avaa ja lukee käyttäjän syötteen mukaisen tiedoston. Kutsuu muistinvaraus aliohjelmää muistin varaamiseksi.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 18 of file linkitettylista.c.

```

18     {
19         FILE *Tiedosto = NULL;
20         char aRivi[LEN] = "";
21         char *p1 = NULL, *p2 = NULL;
22         int iOtsikko = 0;
23         int iMaara = 0;
24
25         /* Tiedoston avaaminen. */
26
27         if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
28             perror("Tiedoston avaaminen epäonnistui, lopetetaan");
29             exit(0);
30         }
31
32         /* Tiedoston lukeminen */
33
34         while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
35
36             aRivi[strlen(aRivi) - 1] = '\0';
37
38             if (iOtsikko == 0) {
39                 iOtsikko = 1;
40                 continue;
41             }
42
43             if ((p1 = strtok(aRivi, ";")) == NULL) {
44                 perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
45                 exit(0);
46             }
47
48             if ((p2 = strtok(NULL, ";")) == NULL) {
49                 perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
50                 exit(0);
51             }
52
53             iMaara = atoi(p2);
54
55             /* Muistin varaaminen ja linkitetyn listan luominen. */
56
57             pAlku = varaaMuistia(pAlku, p1, iMaara);
58         }
59         /* Tiedoston sulkeminen. */
60         fclose(Tiedosto);
61         printf("Tiedosto '%s' luettu.\n", pNimi);

```

```

62     return (pAlku);
63 }

```

4.5.1.4 vapautaMuisti()

```

TIEDOT* vapautaMuisti (
    TIEDOT * pAlku )

```

Vapauttaa linkitetyn listan varaaman muistin.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

pAlku Osoitin, joka on NULL, koska lista on tyhja.

Definition at line 110 of file linkitettylista.c.

```

110 {
111     /* Muistin vapauttaminen. */
112     TIEDOT *ptr = pAlku;
113     while (ptr != NULL) {
114         pAlku = ptr->pSeuraava;
115         free(ptr);
116         ptr = pAlku;
117     }
118     pAlku = NULL;
119     return (pAlku);
120 }

```

4.5.1.5 varaaMuistia()

```

TIEDOT* varaaMuistia (
    TIEDOT * pAlku,
    char * pNimi,
    int iMaara )

```

Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Merkkijono solmun nimeksi.
<i>iMaara</i>	Luku, joka asetetaan solmuun maaraksi.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 73 of file linkitettylista.c.

```

73                                     {
74     TIEDOT *pUusi = NULL;
75     TIEDOT *ptr = NULL;
76
77     /* Muistin varaaminen. */
78
79     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
80         perror("Muistin varaus epäonnistui, lopetetaan");
81         exit(0);
82     }
83
84     /* Kaksisuuntaisen linkitetyn listan luominen. */
85
86     strcpy(pUusi->aSukunimi, pNimi);
87     pUusi->iYhteensa = iMaara;
88     pUusi->pSeuraava = NULL;
89
90     if (pAlku == NULL) {
91         pUusi->pEdellinen = NULL;
92         pAlku = pUusi;
93     } else {
94         ptr = pAlku;
95         while (ptr->pSeuraava != NULL) {
96             ptr = ptr->pSeuraava;
97         }
98         ptr->pSeuraava = pUusi;
99         pUusi->pEdellinen = ptr;
100     }
101     return (pAlku);
102 }

```

4.6 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/linkitettylista.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [tiedot](#)

Typedefs

- typedef struct [tiedot](#) [TIEDOT](#)

Functions

- [TIEDOT * lue](#) (char *pNimi, [TIEDOT](#) *pAlku)
Lukee tiedoston.
- [TIEDOT * varaaMuistia](#) ([TIEDOT](#) *pAlku, char *pSukunimi, int iMaara)
Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.
- [TIEDOT * vapautaMuisti](#) ([TIEDOT](#) *pAlku)
Vapauttaa linkitetyn listan varaaman muistin.
- void [kirjoitaTiedostoAlustaLoppuun](#) (char *pKirjoitaTiedostoNimi, [TIEDOT](#) *pAlku)
Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.
- void [kirjoitaTiedostoLopustaAlkuun](#) (char *pKirjoitaTiedostoNimi, [TIEDOT](#) *pAlku)
Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.

4.6.1 Typedef Documentation

4.6.1.1 TIEDOT

```
typedef struct tiedot TIEDOT
```

4.6.2 Function Documentation

4.6.2.1 kirjoitaTiedostoAlustaLoppuun()

```
void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku )
```

Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 129 of file linkitettylista.c.

```
129                                     {
130     FILE *Tiedosto = NULL;
131     TIEDOT *ptr = NULL;
132
133     if (pAlku == NULL) {
134         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
135         return;
136     }
137
138     // Kirjoitus tiedoston avaaminen
139     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
140         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
141         exit(0);
142     }
143
144     // Tiedostoon kirjoittaminen
145     ptr = pAlku;
146     while (ptr != NULL) {
147         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
148         ptr = ptr->pSeuraava;
149     }
150
151     // Tiedoston sulkeminen
152     fclose(Tiedosto);
153     printf("Tiedosto %s kirjoitettu.\n", pNimi);
154     return;
155 }
```

4.6.2.2 kirjoitaTiedostoLopustaAlkuun()

```
void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku )
```

Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 164 of file linkitettylista.c.

```
164
165     FILE *Tiedosto = NULL;
166     TIEDOT *ptr = pAlku;
167
168     if (pAlku == NULL) {
169         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
170         return;
171     }
172
173     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
174         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
175         exit(0);
176     }
177
178     // Mennään listan loppuun.
179     while (ptr->pSeuraava != NULL) {
180         ptr = ptr->pSeuraava;
181     }
182
183     // Lopusta alkuun, kirjoitetaan tiedostoon.
184     while (ptr != NULL) {
185         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
186         ptr = ptr->pEdellinen;
187     }
188
189     fclose(Tiedosto);
190     printf("Tiedosto %s kirjoitettu.\n", pNimi);
191     return;
192 }
```

4.6.2.3 lue()

```
TIEDOT* lue (
    char * pNimi,
    TIEDOT * pAlku )
```

Lukee tiedoston.

Avaa ja lukee käyttäjän syötteen mukaisen tiedoston. Kutsuu muistinvaraus aliohjelmää muistin varaamiseksi.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 18 of file linkitettylista.c.

```

18                                     {
19     FILE *Tiedosto = NULL;
20     char aRivi[LEN] = "";
21     char *p1 = NULL, *p2 = NULL;
22     int iOtsikko = 0;
23     int iMaara = 0;
24
25     /* Tiedoston avaaminen. */
26
27     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
28         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
29         exit(0);
30     }
31
32     /* Tiedoston lukeminen */
33
34     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
35
36         aRivi[strlen(aRivi) - 1] = '\0';
37
38         if (iOtsikko == 0) {
39             iOtsikko = 1;
40             continue;
41         }
42
43         if ((p1 = strtok(aRivi, ";")) == NULL) {
44             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
45             exit(0);
46         }
47
48         if ((p2 = strtok(NULL, ";")) == NULL) {
49             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
50             exit(0);
51         }
52
53         iMaara = atoi(p2);
54
55         /* Muistin varaaminen ja linkitetyn listan luominen. */
56         pAlku = varaaMuistia(pAlku, p1, iMaara);
57     }
58     /* Tiedoston sulkeminen. */
59     fclose(Tiedosto);
60     printf("Tiedosto '%s' luettu.\n", pNimi);
61     return (pAlku);
62 }
63

```

4.6.2.4 vapautaMuisti()

```

TIEDOT* vapautaMuisti (
    TIEDOT * pAlku )

```

Vapauttaa linkitetyn listan varaaman muistin.

Parameters

pAlku	Osoitin linkitetyn listan alkuun.
-------	-----------------------------------

Returns

pAlku Osoitin, joka on NULL, koska lista on tyhja.

Definition at line 110 of file linkitettylista.c.

```

110                                     {
111     /* Muistin vapauttaminen. */
112     TIEDOT *ptr = pAlku;
113     while (ptr != NULL) {
114         pAlku = ptr->pSeuraava;
115         free(ptr);
116         ptr = pAlku;
117     }
118     pAlku = NULL;
119     return (pAlku);
120 }

```

4.6.2.5 varaaMuistia()

```

TIEDOT* varaaMuistia (
    TIEDOT * pAlku,
    char * pNimi,
    int iMaara )

```

Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Merkkijono solmun nimeksi.
<i>iMaara</i>	Luku, joka asetetaan solmuun maaraksi.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 73 of file linkitettylista.c.

```

73                                     {
74     TIEDOT *pUusi = NULL;
75     TIEDOT *ptr = NULL;
76
77     /* Muistin varaaminen. */
78
79     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
80         perror("Muistin varaus epäonnistui, lopetetaan");
81         exit(0);
82     }
83
84     /* Kaksisuuntaisen linkitetyn listan luominen. */
85
86     strcpy(pUusi->aSukunimi, pNimi);
87     pUusi->iYhteensa = iMaara;
88     pUusi->pSeuraava = NULL;
89
90     if (pAlku == NULL) {
91         pUusi->pEdellinen = NULL;
92         pAlku = pUusi;
93     } else {
94         ptr = pAlku;
95         while (ptr->pSeuraava != NULL) {
96             ptr = ptr->pSeuraava;
97         }
98         ptr->pSeuraava = pUusi;
99         pUusi->pEdellinen = ptr;
100     }
101     return (pAlku);
102 }

```


4.7 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/paaohjelma.c File Reference

```
#include "binaaripuu.h"
#include "linkitettylista.h"
#include "yleiset.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- int `main` (void)

Paaohjelma, pyorittaa koko ohjelmaa.

4.7.1 Function Documentation

4.7.1.1 main()

```
int main (
    void )
```

Paaohjelma, pyorittaa koko ohjelmaa.

Returns

int Palauttaa aina 0, kun ohjelma tulee päätökseen.

Definition at line 13 of file paaohjelma.c.

```
13     {
14         int iValinta = 0;
15
16         do {
17             iValinta = valikko();
18             if (iValinta == 1) {
19                 mainLista();
20             } else if (iValinta == 2) {
21                 mainBinaaripuu();
22             } else if (iValinta == 0) {
23                 printf("Lopetetaan.\n");
24             } else {
25                 printf("Tuntematon valinta, yritä uudestaan.\n");
26             }
27             printf("\n");
28         } while (iValinta != 0);
29
30         printf("Kiitos ohjelman käytöstä.\n");
31         return (0);
32     }
```

4.8 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/punamustapuu.c File Reference

```
#include "binaaripuu.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- [RBSOLMU * varaaMuistiaRB](#) (char *pNimi, int iArvo)
L10 / Punamustapuu Vaatii oman structin takia omat aliohjelmat, jotka ovat [binaaripuu.c](#) tiedoston aliohjelmiä muokattuina.
- [RBSOLMU * vapautaMuistiRB](#) (RBSOLMU *pJuuriSolmu)
- [RBSOLMU * lisaaRBSolmu](#) (RBSOLMU *pJuuriSolmu, [RBSOLMU *pUusi](#))
- [RBSOLMU * luoRBPuu](#) (RBSOLMU *pJuuriSolmu, char *pNimi)
- void [kierraVasemmalle](#) (RBSOLMU **pJuuriSolmu, [RBSOLMU *pSolmu](#))
- void [kierraOikealle](#) (RBSOLMU **pJuuriSolmu, [RBSOLMU *pSolmu](#))
- void [korjaaLisays](#) (RBSOLMU **pJuuriSolmu, [RBSOLMU *pUusi](#))
- void [kirjoitaRBTiedostoon](#) (char *pNimi, [RBSOLMU *pJuuriSolmu](#))
- void [kirjoitaRB](#) (char *pNimi, [RBSOLMU *pJuuriSolmu](#))

4.8.1 Function Documentation

4.8.1.1 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU \*\* pJuuriSolmu,
    RBSOLMU \* pSolmu )
```

Definition at line 130 of file punamustapuu.c.

```
130
131     RBSOLMU \*pVasenLapsi = pSolmu->pVasen;
132     pSolmu->pVasen = pVasenLapsi->pOikea;
133
134     if (pSolmu->pVasen != NULL)
135         pSolmu->pVasen->pVanhempi = pSolmu;
136
137     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
138
139     if (pSolmu->pVanhempi == NULL)
140         *pJuuriSolmu = pVasenLapsi;
141     else if (pSolmu == pSolmu->pVanhempi->pVasen)
142         pSolmu->pVanhempi->pVasen = pVasenLapsi;
143     else
144         pSolmu->pVanhempi->pOikea = pVasenLapsi;
145
146     pVasenLapsi->pOikea = pSolmu;
147     pSolmu->pVanhempi = pVasenLapsi;
148 }
```

4.8.1.2 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 109 of file punamustapuu.c.

```
109 {
110     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
111     pSolmu->pOikea = pOikeaLapsi->pVasen;
112
113     if (pSolmu->pOikea != NULL)
114         pSolmu->pOikea->pVanhempi = pSolmu;
115
116     // Solmun oikea lapsi nousee ylös puussa
117     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
118
119     if (pSolmu->pVanhempi == NULL)
120         *pJuurisolmu = pOikeaLapsi;
121     else if (pSolmu == pSolmu->pVanhempi->pVasen)
122         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
123     else
124         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
125
126     pOikeaLapsi->pVasen = pSolmu;
127     pSolmu->pVanhempi = pOikeaLapsi;
128 }
```

4.8.1.3 kirjoitaRB()

```
void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 222 of file punamustapuu.c.

```
222 {
223     /*if (pJuurisolmu == NULL) {
224         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
225         return;
226     }*/
227     if (pJuurisolmu != NULL) {
228         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
229         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
230         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
231     }
232     return;
233 }
```

4.8.1.4 kirjoitaRBTiedostoon()

```
void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 201 of file punamustapuu.c.

```
201 {
202     FILE *Tiedosto = NULL;
203
204     if (pJuurisolmu == NULL) {
205         return;
206     }
207
208     /* Tiedoston avaaminen. */
209     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
```

```

210         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
211         exit(0);
212     }
213     /* Tiedostoon kirjoittaminen. */
214     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
215
216     /* Tiedoston sulkeminen. */
217     fclose(Tiedosto);
218     printf("Tiedosto '%s' kirjoitettu.\n", pNimi);
219     return;
220 }

```

4.8.1.5 korjaaLisays()

```

void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi )

```

Definition at line 150 of file punamustapuu.c.

```

150     {
151         // Ilman p-alkua, koska pVanhempi->pVanhempi oli sekava
152         RBSOLMU *vanhempi = NULL;
153         RBSOLMU *seta = NULL;
154         RBSOLMU *isoVanhempi = NULL;
155
156         while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
157             vanhempi = pUusi->pVanhempi;
158             isoVanhempi = vanhempi->pVanhempi;
159
160             if (vanhempi == isoVanhempi->pVasen) {
161                 seta = isoVanhempi->pOikea;
162                 if (seta != NULL && seta->iVariBitti == 0) {
163                     vanhempi->iVariBitti = 1;
164                     seta->iVariBitti = 1;
165                     isoVanhempi->iVariBitti = 0;
166                     pUusi = isoVanhempi;
167                 } else {
168                     if (pUusi == vanhempi->pOikea) {
169                         pUusi = vanhempi;
170                         kierraVasemmalle(pJuurisolmu, pUusi);
171                         vanhempi = pUusi->pVanhempi;
172                     }
173                     vanhempi->iVariBitti = 1;
174                     isoVanhempi->iVariBitti = 0;
175                     kierraOikealle(pJuurisolmu, isoVanhempi);
176                 }
177             } else {
178                 seta = isoVanhempi->pVasen;
179                 if (seta != NULL && seta->iVariBitti == 0) {
180                     vanhempi->iVariBitti = 1;
181                     seta->iVariBitti = 1;
182                     isoVanhempi->iVariBitti = 0;
183                     pUusi = isoVanhempi;
184                 } else {
185                     if (pUusi == vanhempi->pVasen) {
186                         pUusi = vanhempi;
187                         kierraOikealle(pJuurisolmu, pUusi);
188                         vanhempi = pUusi->pVanhempi;
189                     }
190                     vanhempi->iVariBitti = 1;
191                     isoVanhempi->iVariBitti = 0;
192                     kierraVasemmalle(pJuurisolmu, isoVanhempi);
193                 }
194             }
195         }
196     }
197     (*pJuurisolmu)->iVariBitti = 1;
198 }

```

4.8.1.6 lisaaRBSolmu()

```
RBSOLMU* lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi )
```

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 38 of file punamustapuu.c.

```
38                                     {
39     int iVertailu = 0;
40
41     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
42     if (pJuurisolmu == NULL) {
43         return pUusi;
44     }
45
46     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
47
48     // Solmujen lisääminen rekursiivisesti
49     if (pUusi->iArvo < pJuurisolmu->iArvo) {
50         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
51         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
52     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
53         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
54         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
55     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
56         if (iVertailu < 0) {
57             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
58             pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
59         } else if (iVertailu > 0) {
60             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
61             pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
62         }
63     }
64     return (pJuurisolmu);
65 }
66
67 }
```

4.8.1.7 luoRBPuu()

```
RBSOLMU* luoRBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi )
```

Definition at line 69 of file punamustapuu.c.

```
69                                     {
70     FILE *Tiedosto = NULL;
71     char aRivi[LEN] = "";
72     int iOtsikko = 0;
73     char *p1 = NULL, *p2 = NULL;
74     int iArvo = 0;
75
76     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
77         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
78         exit(0);
79     }
80
81     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
82         aRivi[strlen(aRivi) - 1] = '\0';
83         if (iOtsikko == 0) {
84             iOtsikko++;
85             continue;
86         }
87
88         if ((p1 = strtok(aRivi, ";")) == NULL) {
89             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
90             exit(0);
91         }
92
93         if ((p2 = strtok(NULL, ";")) == NULL) {
94             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
95             exit(0);
96         }
97     }
```

```

97
98     iArvo = atoi(p2);
99
100     RBSOLMU *pUusi = varaaMuistiaRB(pl, iArvo);
101     pJuuriSolmu = lisaaRBSolmu(pJuuriSolmu, pUusi);
102     korjaaLisays(&pJuuriSolmu, pUusi);
103 }
104
105 fclose(Tiedosto);
106 return (pJuuriSolmu);
107 }

```

4.8.1.8 vapautaMuistiRB()

```

RBSOLMU* vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu )

```

Definition at line 28 of file punamustapuu.c.

```

28                                     {
29     if (pJuuriSolmu != NULL) {
30         vapautaMuistiRB(pJuuriSolmu->pVasen);
31         vapautaMuistiRB(pJuuriSolmu->pOikea);
32         free(pJuuriSolmu);
33     }
34     pJuuriSolmu = NULL;
35     return (pJuuriSolmu);
36 }

```

4.8.1.9 varaaMuistiaRB()

```

RBSOLMU* varaaMuistiaRB (
    char * pNimi,
    int iArvo )

```

L10 / Punamustapuu Vaatii oman structin takia omat aliohjelmat, jotka ovat [binaaripuu.c](#) tiedoston aliohjelmia muokattuina.

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Definition at line 11 of file punamustapuu.c.

```

11                                     {
12     RBSOLMU *pUusi = NULL;
13
14     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
15         perror("Muistin varaus epäonnistui, lopetetaan");
16         exit(0);
17     }
18
19     pUusi->iVariBitti = 0; // uudet aina punaisia
20     strcpy(pUusi->aNimi, pNimi);
21     pUusi->iArvo = iArvo;
22     pUusi->pVasen = NULL;
23     pUusi->pOikea = NULL;
24     pUusi->pVanhempi = NULL;
25     return (pUusi);
26 }

```

4.9 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/yleiset.c File Reference

```
#include "yleiset.h"
#include "linkitettylista.h"
#include "binaaripuu.h"
#include "haut.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- void [mainLista](#) ()
Tekee käyttäjän valinnan perusteella listan toiminnot.
- void [mainBinaaripuu](#) ()
Tekee käyttäjän valinnan perusteella binääripuun toiminnot.
- int [valikko](#) (void)
Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binaaripuun.
- int [listaValikko](#) (void)
Tulostaa lista valikon ja kysyy käyttäjän valinnan.
- int [binaaripuuValikko](#) (void)
Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.
- char * [kysyNimi](#) (char *pPrompti, char *pNimi)
Kysyy tiedoston/haettavan nimen.
- int [kysyArvo](#) (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsiä/poistaa puusta.

4.9.1 Function Documentation

4.9.1.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.

Parameters

<i>iValinta</i>	Käyttäjän antama syöte vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 202 of file yleiset.c.

```

202         {
203     int iValinta = 0;
204     printf("\nValitse haluamasi toiminto:\n");
205     printf("1) Luo puu\n");
206     printf("2) Kirjoita AVL puu tiedostoon\n");
207     printf("3) Syvyyshaku\n");
208     printf("4) Leveyshaku\n");
209     printf("5) Tyhjennä puu\n");
210     printf("6) Poista solmu\n");
211     printf("7) Binäärihaku AVL puusta\n");
212     printf("8) Punamustapuu\n");
213     printf("0) Takaisin\n");
214     printf("Anna valintasi: ");
215     scanf("%d", &iValinta);
216     getchar();
217     return iValinta;
218 }

```

4.9.1.2 kysyArvo()

```

int kysyArvo (
    char * pPrompti,
    int iArvo )

```

Kysyy arvon, joka halutaan etsiä/poistaa puusta.

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä.

Returns

int Palauttaa käyttäjän antaman arvon.

Definition at line 241 of file yleiset.c.

```

241         {
242     printf("%s", pPrompti);
243     scanf("%d", &iArvo);
244     getchar();
245     return (iArvo);
246 }

```

4.9.1.3 kysyNimi()

```

char* kysyNimi (
    char * pPrompti,
    char * pNimi )

```

Kysyy tiedoston/haettavan nimen.

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna kirjoitettavan tiedoston nimi: "
<i>pNimi</i>	Tiedoston nimi/haettava nimi, jota halutaan käyttää.

Returns

char Palauttaa kayttajan antaman merkkijonon.

Definition at line 227 of file yleiset.c.

```

227                                     {
228     printf("%s", pPrompti);
229     scanf("%s", pNimi);
230     getchar();
231     return (pNimi);
232 }
```

4.9.1.4 listaValikko()

```

int listaValikko (
    void )
```

Tulostaa lista valikon ja kysyy kayttajan valinnan.

Parameters

<i>iValinta</i>	Kayttajan antama syote vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa kayttajan valinnan.

Definition at line 182 of file yleiset.c.

```

182     {
183     int iValinta = 0;
184     printf("\nValitse haluamasi toiminto:\n");
185     printf("1) Lue tiedosto\n");
186     printf("2) Kirjoita tiedosto alusta loppuun\n");
187     printf("3) Kirjoita tiedosto lopusta alkuun\n");
188     printf("4) Tyhjennä taulukko\n");
189     printf("0) Takaisin\n");
190     printf("Anna valintasi: ");
191     scanf("%d", &iValinta);
192     getchar();
193     return iValinta;
194 }
```

4.9.1.5 mainBinaaripuu()

```

void mainBinaaripuu ( )
```

Tekee käyttäjän valinnan perusteella binääripuun toiminnot.

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset binääripuuhun liittyvät toiminnot.

Definition at line 70 of file yleiset.c.

```

70     {
71     char aNimiLuettava[LEN] = "";
72     char aNimiKirjoitettava[LEN] = "";
73     char aHaettavaNimi[LEN] = "";
74     PUU *pJuuriSolmu = NULL;
75     RBSOLMU *pRBJuuri = NULL;
```

```

76     int iValinta = 0;
77     int iArvo = 0;
78
79     do {
80         iValinta = binaaripuuValikko();
81         if (iValinta == 1) {
82             kysyNimi("Tiedoston nimi: ", aNimiLuettava);
83             pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
84
85         } else if (iValinta == 2) {
86             if (pJuuriSolmu == NULL) {
87                 printf("Luo binääripuu ennen sen kirjoittamista.\n");
88             } else {
89                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
90                 kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
91             }
92
93         } else if (iValinta == 3) {
94             if (pJuuriSolmu == NULL) {
95                 printf("Luo binääripuu ennen syvyyshakua.\n");
96             } else {
97                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
98                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
99                 tarkistaLoytvykoSyvyyshauulla(aNimiKirjoitettava, pJuuriSolmu, iArvo);
100            }
101
102        } else if (iValinta == 4) {
103            if (pJuuriSolmu == NULL) {
104                printf("Luo binääripuu ennen leveyshakua.\n");
105            } else {
106                kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
107                kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
108                leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
109            }
110
111        } else if (iValinta == 5) {
112            pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
113            printf("Muisti vapautettu.\n");
114
115        } else if (iValinta == 6) {
116            if (pJuuriSolmu == NULL) {
117                printf("Luo binääripuu ennen poistoa.\n");
118            } else {
119                kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
120                pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);
121            }
122
123        } else if (iValinta == 7) {
124            if (pJuuriSolmu == NULL) {
125                printf("Luo binääripuu ennen binäärihakua.\n");
126            } else {
127                kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
128                iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
129                iArvo = binaarihaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
130                if (iArvo == 0) {
131                    printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
132                }
133            }
134
135        } else if (iValinta == 8) {
136            // Aino säätää ja testaa
137            kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
138            kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
139            pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
140            kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
141            pRBJuuri = vapautaMuistiRB(pRBJuuri);
142
143        } else if (iValinta == 0) {
144            printf("Palataan takaisin päävalikkoon.\n");
145
146        } else {
147            printf("Tuntematon valinta, yritä uudestaan.\n");
148        }
149    } while (iValinta != 0);
150
151    /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
152    * käyttäjä ei muista sitä tehdä. */
153
154    pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
155    return;
156 }

```

4.9.1.6 mainLista()

```
void mainLista ( )
```

Tekee käyttäjän valinnan perusteella listan toiminnot.

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset listaan liittyvät toiminnot.

Definition at line 17 of file yleiset.c.

```
17     {
18         char aNimiLuettava[LEN] = "";
19         char aNimiKirjoitettava[LEN] = "";
20         int iValinta = 0;
21         TIEDOT *pAlku = NULL;
22
23         do {
24             iValinta = listaValikko();
25             if (iValinta == 1) {
26                 kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
27                 pAlku = lue(aNimiLuettava, pAlku);
28
29             } else if (iValinta == 2) {
30                 if (pAlku == NULL) {
31                     printf("Lue tiedosto ennen kirjoittamista.\n");
32                 } else {
33                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
34                     kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
35                 }
36
37             } else if (iValinta == 3) {
38                 if (pAlku == NULL) {
39                     printf("Lue tiedosto ennen kirjoittamista.\n");
40                 } else {
41                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
42                     kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
43                 }
44
45             } else if (iValinta == 4) {
46                 pAlku = vapautaMuisti(pAlku);
47                 printf("Muisti vapautettu.\n");
48
49             } else if (iValinta == 0) {
50                 printf("Palataan takaisin päävalikkoon.\n");
51
52             } else {
53                 printf("Tuntematon valinta, yritä uudestaan.\n");
54             }
55         } while (iValinta != 0);
56
57         /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
58         * käyttäjä ei muista sitä tehdä. */
59
60         pAlku = vapautaMuisti(pAlku);
61         return;
62     }
```

4.9.1.7 valikko()

```
int valikko (
    void )
```

Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binaaripuun.

Parameters

<i>iValinta</i>	Käyttäjän antama syöte.
-----------------	-------------------------

Returns

int Palauttaa kayttajan valinnan.

Definition at line 164 of file yleiset.c.

```
164     {
165         int iValinta = 0;
166         printf("Valitse haluamasi toiminto:\n");
167         printf("1) Lista\n");
168         printf("2) Binääripuu\n");
169         printf("0) Lopeta\n");
170         printf("Anna valintasi: ");
171         scanf("%d", &iValinta);
172         getchar();
173         return iValinta;
174     }
```

4.10 /home/sofia_toropainen/CT60A2600-Tiimi-A/src/yleiset.h File Reference

Macros

- #define [LEN](#) 50

Functions

- void [mainLista](#) ()
Tekee käyttäjän valinnan perusteella listan toiminnot.
- void [mainBinaaripuu](#) ()
Tekee käyttäjän valinnan perusteella binääripuun toiminnot.
- int [valikko](#) (void)
Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binaaripuun.
- int [listaValikko](#) (void)
Tulostaa lista valikon ja kysyy kayttajan valinnan.
- int [binaaripuuValikko](#) (void)
Tulostaa binaariluku valikon ja kysyy kayttajalta valinnan.
- char * [kysyNimi](#) (char *pPrompti, char *pNimi)
Kysyy tiedoston/haettavan nimen.
- int [kysyArvo](#) (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsia/poistaa puusta.

4.10.1 Macro Definition Documentation

4.10.1.1 LEN

```
#define LEN 50
```

Definition at line 4 of file yleiset.h.

4.10.2 Function Documentation

4.10.2.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.

Parameters

<i>iValinta</i>	Käyttäjän antama syöte vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 202 of file yleiset.c.

```
202     {
203         int iValinta = 0;
204         printf("\nValitse haluamasi toiminto:\n");
205         printf("1) Luo puu\n");
206         printf("2) Kirjoita AVL puu tiedostoon\n");
207         printf("3) Syvyyshaku\n");
208         printf("4) Leveyshaku\n");
209         printf("5) Tyhjennä puu\n");
210         printf("6) Poista solmu\n");
211         printf("7) Binäärihaku AVL puusta\n");
212         printf("8) Punamustapuu\n");
213         printf("0) Takaisin\n");
214         printf("Anna valintasi: ");
215         scanf("%d", &iValinta);
216         getchar();
217         return iValinta;
218     }
```

4.10.2.2 kysyArvo()

```
int kysyArvo (
    char * pPrompti,
    int iArvo )
```

Kysyy arvon, joka halutaan etsiä/poistaa puusta.

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä.

Returns

int Palauttaa kayttajan antaman arvon.

Definition at line 241 of file yleiset.c.

```
241                                     {
242     printf("%s", pPrompti);
243     scanf("%d", &iArvo);
244     getchar();
245     return (iArvo);
246 }
```

4.10.2.3 kysyNimi()

```
char* kysyNimi (
    char * pPrompti,
    char * pNimi )
```

Kysyy tiedoston/haettavan nimen.

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna kirjoitettavan tiedoston nimi: "
<i>pNimi</i>	Tiedoston nimi/haettava nimi, jota halutaan kayttaa.

Returns

char Palauttaa kayttajan antaman merkkijonon.

Definition at line 227 of file yleiset.c.

```
227                                     {
228     printf("%s", pPrompti);
229     scanf("%s", pNimi);
230     getchar();
231     return (pNimi);
232 }
```

4.10.2.4 listaValikko()

```
int listaValikko (
    void )
```

Tulostaa lista valikon ja kysyy kayttajan valinnan.

Parameters

<i>iValinta</i>	Kayttajan antama syote vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 182 of file yleiset.c.

```
182     {
183         int iValinta = 0;
184         printf("\nValitse haluamasi toiminto:\n");
185         printf("1) Lue tiedosto\n");
186         printf("2) Kirjoita tiedosto alusta loppuun\n");
187         printf("3) Kirjoita tiedosto lopusta alkuun\n");
188         printf("4) Tyhjennä taulukko\n");
189         printf("0) Takaisin\n");
190         printf("Anna valintasi: ");
191         scanf("%d", &iValinta);
192         getchar();
193         return iValinta;
194     }
```

4.10.2.5 mainBinaaripuu()

```
void mainBinaaripuu ( )
```

Tekee käyttäjän valinnan perusteella binääripuun toiminnot.

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset binääripuuhun liittyvät toiminnot.

Definition at line 70 of file yleiset.c.

```
70     {
71         char aNimiLuettava[LEN] = "";
72         char aNimiKirjoitettava[LEN] = "";
73         char aHaettavaNimi[LEN] = "";
74         PUU *pJuuriSolmu = NULL;
75         RBSOLMU *pRBJuuri = NULL;
76         int iValinta = 0;
77         int iArvo = 0;
78
79         do {
80             iValinta = binaaripuuValikko();
81             if (iValinta == 1) {
82                 kysyNimi("Tiedoston nimi: ", aNimiLuettava);
83                 pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
84             } else if (iValinta == 2) {
85                 if (pJuuriSolmu == NULL) {
86                     printf("Luo binääripuu ennen sen kirjoittamista.\n");
87                 } else {
88                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
89                     kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
90                 }
91             } else if (iValinta == 3) {
92                 if (pJuuriSolmu == NULL) {
93                     printf("Luo binääripuu ennen syvyyshakua.\n");
94                 } else {
95                     iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
96                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
97                     tarkistaLoytyykoSyvyyshaula(aNimiKirjoitettava, pJuuriSolmu, iArvo);
98                 }
99             } else if (iValinta == 4) {
100                 if (pJuuriSolmu == NULL) {
101                     printf("Luo binääripuu ennen leveyshakua.\n");
102                 } else {
103                     kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
104                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
105                     leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
106                 }
107             } else if (iValinta == 5) {
108                 pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
109                 printf("Muisti vapautettu.\n");
110             } else if (iValinta == 6) {
111                 if (pJuuriSolmu == NULL) {
```

```

117         printf("Luo binääripuu ennen poistoa.\n");
118     } else {
119         kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
120         pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);
121     }
122
123     } else if (iValinta == 7) {
124         if (pJuuriSolmu == NULL) {
125             printf("Luo binääripuu ennen binäärihakua.\n");
126         } else {
127             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
128             iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
129             iArvo = binaarihaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
130             if (iArvo == 0) {
131                 printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
132             }
133         }
134     }
135     } else if (iValinta == 8) {
136         // Aino säätää ja testaa
137         kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
138         kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
139         pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
140         kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
141         pRBJuuri = vapautaMuistiRB(pRBJuuri);
142     }
143     } else if (iValinta == 0) {
144         printf("Palataan takaisin päävalikkoon.\n");
145     }
146     } else {
147         printf("Tuntematon valinta, yritä uudestaan.\n");
148     }
149 } while (iValinta != 0);
150
151 /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
152 * käyttäjä ei muista sitä tehdä. */
153
154 pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
155 return;
156 }

```

4.10.2.6 mainLista()

```
void mainLista ( )
```

Tekee käyttäjän valinnan perusteella listan toiminnot.

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset listaan liittyvät toiminnot.

Definition at line 17 of file yleiset.c.

```

17     {
18         char aNimiLuettava[LEN] = "";
19         char aNimiKirjoitettava[LEN] = "";
20         int iValinta = 0;
21         TIEDOT *pAlku = NULL;
22
23         do {
24             iValinta = listaValikko();
25             if (iValinta == 1) {
26                 kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
27                 pAlku = lue(aNimiLuettava, pAlku);
28             }
29             } else if (iValinta == 2) {
30                 if (pAlku == NULL) {
31                     printf("Lue tiedosto ennen kirjoittamista.\n");
32                 } else {
33                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
34                     kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
35                 }
36             }
37             } else if (iValinta == 3) {
38                 if (pAlku == NULL) {
39                     printf("Lue tiedosto ennen kirjoittamista.\n");
40                 } else {
41                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
42                     kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
43                 }

```



```
44
45     } else if (iValinta == 4) {
46         pAlku = vapautaMuisti(pAlku);
47         printf("Muisti vapautettu.\n");
48
49     } else if (iValinta == 0) {
50         printf("Palataan takaisin päävalikkoon.\n");
51
52     } else {
53         printf("Tuntematon valinta, yritä uudestaan.\n");
54     }
55     } while (iValinta != 0);
56
57     /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
58     * käyttäjä ei muista sitä tehdä. */
59
60     pAlku = vapautaMuisti(pAlku);
61     return;
62 }
```

4.10.2.7 valikko()

```
int valikko (
    void )
```

Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binaaripuun.

Parameters

<i>iValinta</i>	Kayttajan antama syote.
-----------------	-------------------------

Returns

int Palauttaa kayttajan valinnan.

Definition at line 164 of file yleiset.c.

```
164     {
165         int iValinta = 0;
166         printf("Valitse haluamasi toiminto:\n");
167         printf("1) Lista\n");
168         printf("2) Binääripuu\n");
169         printf("0) Lopeta\n");
170         printf("Anna valintasi: ");
171         scanf("%d", &iValinta);
172         getchar();
173         return iValinta;
174     }
```

